МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №6

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод простых итераций.

Студент гр. 4342 Кринкин Н.К.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Вариант 8. $f(x) = 2x^2 - x^4 - 1 + ln(x)$

Изучить метод простых итераций для решения нелинейных уравнений. Найти корень заданного уравнения с заданной точностью, исследовать зависимость количества итераций от требуемой точности и оценить устойчивость метода к ошибкам в исходных данных.

Основные теоретические положения.

Метод простых итераций решения уравнения $f(x) = 0$ состоит в замене исходного уравнения эквивалентным ему уравнением $x = \varphi(x)$ и построении последовательности $x_{n+1} = \varphi(x_n)$, сходящейся при $n \to \infty$ к точному решению. Достаточные условия сходимости метода простых итераций формулируются теоремой, приведенной [1,2,7].

Рассмотрим один шаг итерационного процесса. Исходя из найденного на предыдущем шаге значения x_{n-1}, вычисляется $y = \varphi(x_{n-1})$. Если $|y - x_{n-1}| > \varepsilon$, то полагается $x_n = y$ и выполняется очередная итерация. Если же $|y - x_{n-1}| < \varepsilon$, то вычисления заканчиваются и за приближенное значение корня принимается величина $x_n = y$. Погрешность результата вычислений зависит от знака производной $\varphi'(x)$: при $\varphi'(x) > 0$ погрешность определения корня составляет $q\varepsilon/1 - q$, а при $\varphi'(x) < 0$ погрешность не превышает ε. Здесь q- число, такое, что $|\varphi'(x)| \leq q < 1$ на отрезке [a,b]. Существование числа q является условием сходимости метода в соответствии с отмеченной выше теоремой.

Для применения метода простых итераций определяющее значение имеет выбор функции $\varphi(x)$ в уравнении $x = \varphi(x)$, эквивалентном исходному. Функцию $\varphi(x)$ необходимо подбирать так, чтобы $|\varphi'(x)| \leq q < 1$. Это обусловливается тем, что если $\varphi'(x) < 0$ на отрезке [a,b], то последовательные приближения $x_n = \varphi(x_{n-1})$ будут колебаться около корня с, если же $\varphi'(x) > 0$,

то последовательные приближения будут сходиться к корню с монотонно. Следует также помнить, что скорость сходимости последовательности $\{x_n\}$ к корню с функции $f(x)$ тем выше, чем выше число q.

Выполнение работы.

1) Нахождение отрезка

   Построив график функции (рис. 2), выберем промежуток [0, 2] в отправной точки. Как видно по графику, отрезок содержит корень уравнения. Возьмем $x_0 = 1.05$ в качестве начального приближения.

2) Построим функцию $\varphi(x)$ путем преобразования:

   $$f(x) = 0 \;=>\; 2x^2 - x^4 - 1 + ln(x) = 0$$

   $$=>\; \varphi(x) = e^{2x^2 - x^4 - 1}$$

   Сузим промежуток, чтобы получить промежуток, на котором выполняется условие $|\varphi'(x)| \leq q < 1$ – получаем промежуток [0.9; 1.1]. Заметим, что наше начальное приближение входит в данный промежуток.

3) Реализуем функцию f(x) и $\varphi(x)$, используя инструменты стандартной библиотеки C++:

```
double origF(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}

double F(double x) {
    return pow(exp(1.0), 2 * pow(x, 2) - pow(x, 4) - 1);
}
```

   Полный код программы представлен в приложении А.

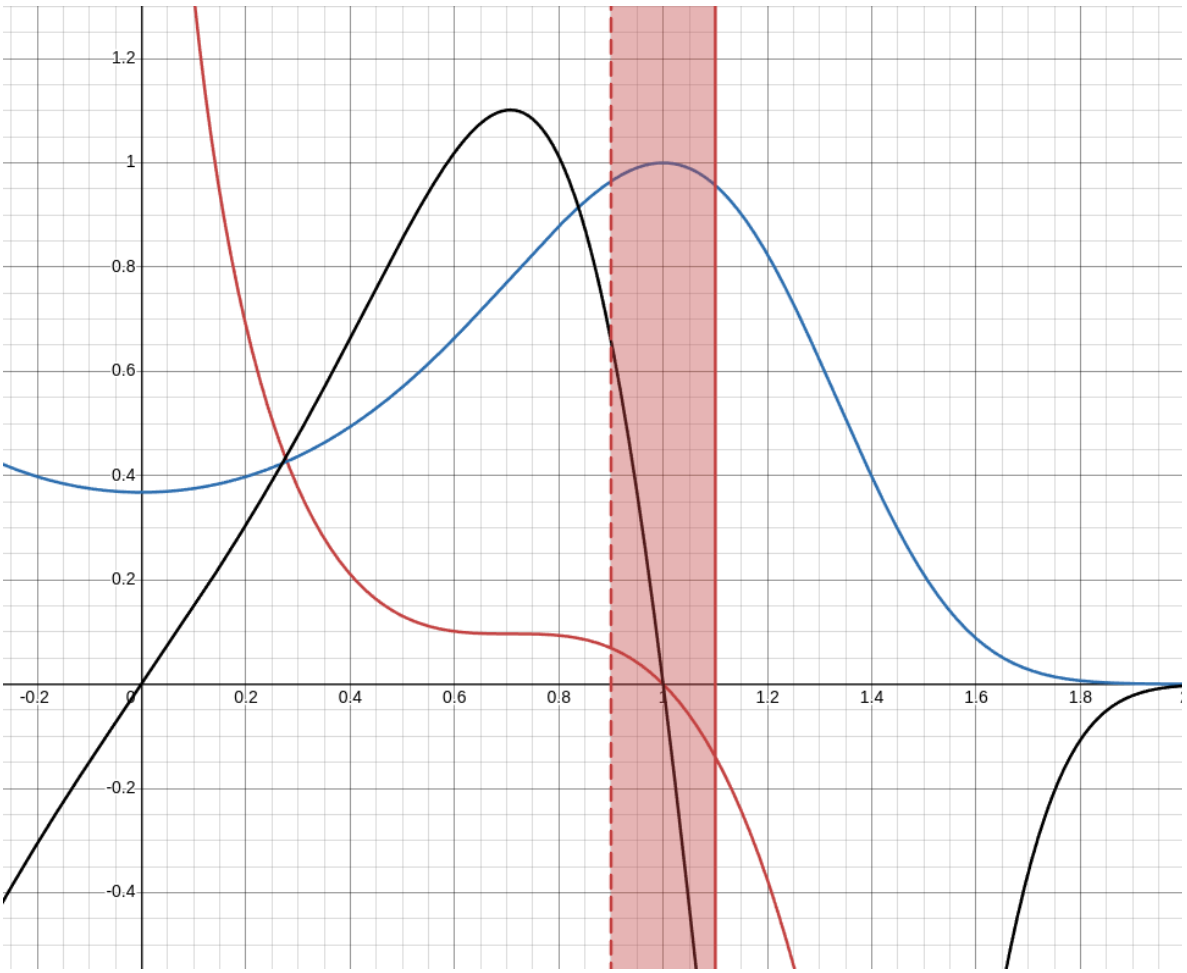

Рисунок 1 - график функции $f(x)$ (красный), $\varphi(x)$ (синий) и $\varphi'(x)$ (черный)
Также красным обозначена область, на которой $|\varphi'(x)| \leq q < 1$

4) В основной файле определим функцию main, которая представляет, которая представляет из себя цикл, перебирающий значение порядка точности ε и выводит результат работы функции ITER с точностью delta

5) Проведены вычисления, построен график (Рисунок 2).

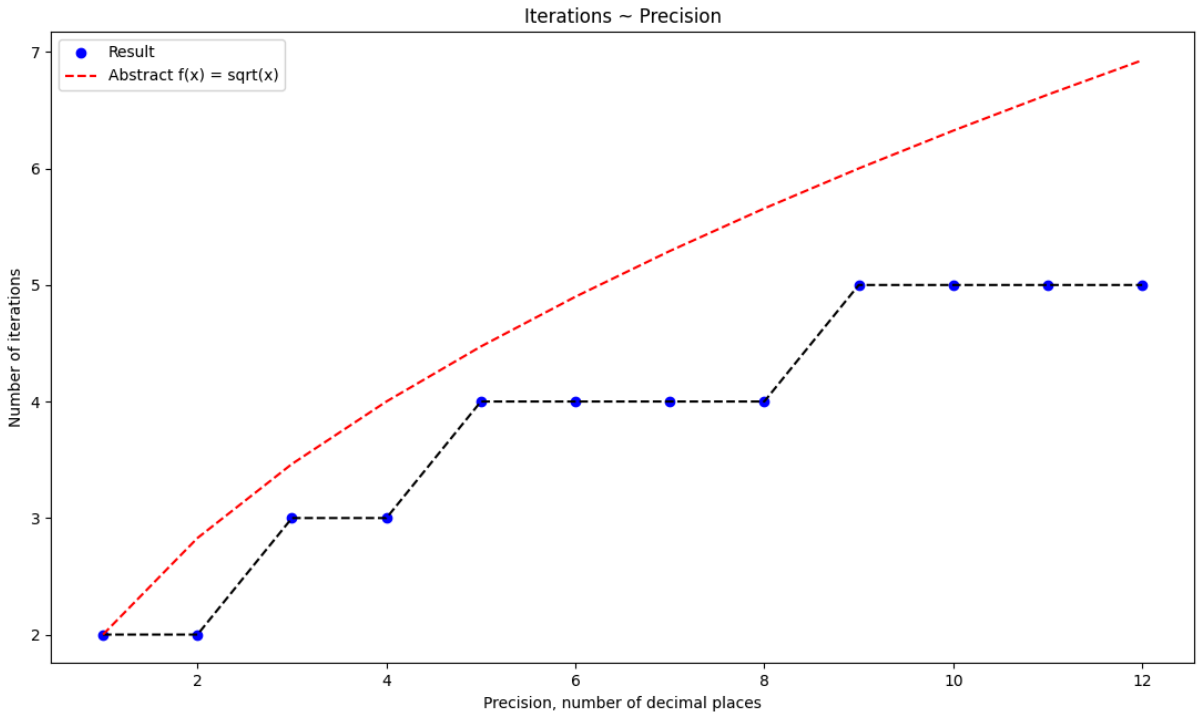

Рисунок 2. График зависимости количества итераций от порядка точности

Если провести исследование графика, можно понять, что порядок точность и количество итераций формируют зависимость квадратного корня.

6) Для исследования чувствительности метода ко входным данным была написана дополнительная программа, проводящая итерацию по начальным приближениям в промежутке от [0.2; 2) с шагом 0.02 и измеряющая количество шагов алгоритма для нахождения корня. В рамках данного исследования была выбрана точность eps = 10^-8, чтобы влияние ошибок входных данных или ошибок в выборе корня были более заметны. Результаты представлены на рисунке 3.

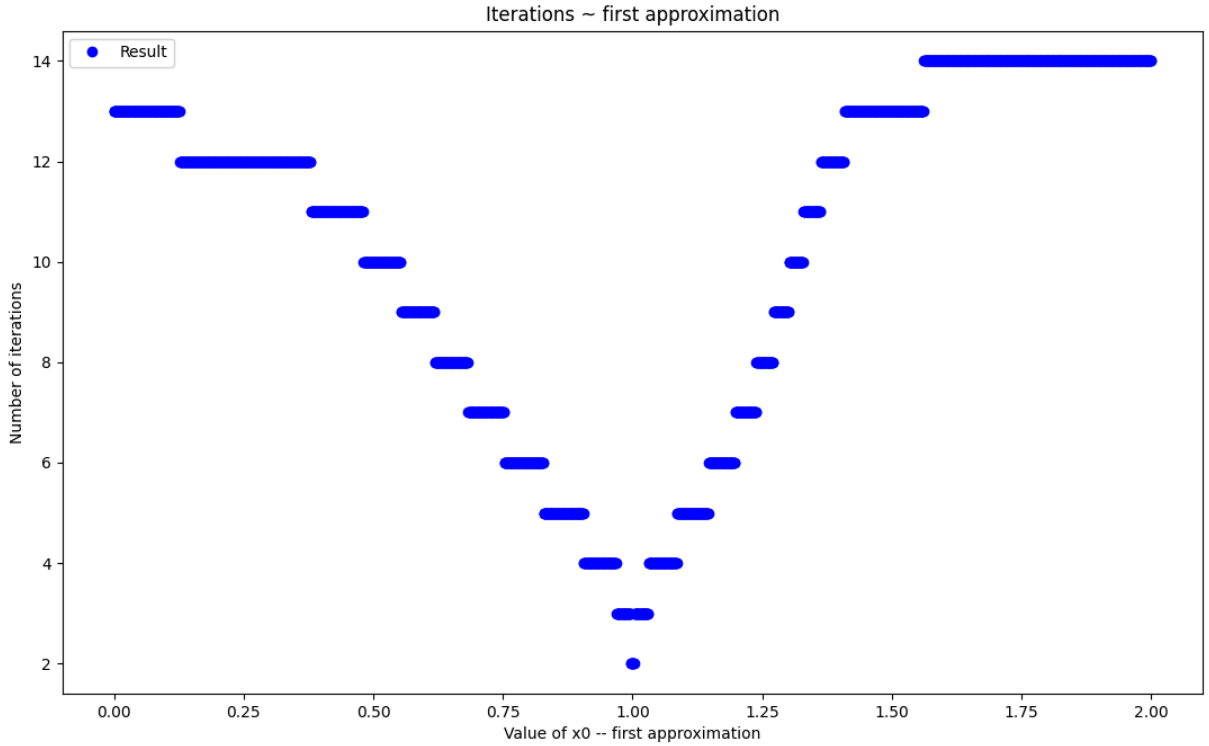

Рисунок 3. График зависимости количества итераций от начального приближения

Исходя из результатов исследования, можно сделать вывод об устойчивости метода отрезков к ошибкам в округлении.

Выводы.

В ходе работы был изучен метод бисекции (половинного деления). Для уравнения $f(x) = 2x^2 - x^4 - 1 + ln(x)$ корень был успешно найден на отрезке [0.85; 1.15] с точностью от 0.1 до 10^-12. Построенный график показал линейную зависимость числа итераций от порядка точности: увеличение точности (уменьшение ε) требует ожидаемого роста количества вычислений, что соответствует теории. Исследование обусловленности показало, что метод хорд является устойчивым к малым ошибкам округления значений функции, однако при грубых возмущениях (большая Delta) процесс может замедлиться или привести к потере точности нахождения корня.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл `precision.cpp`:

```
#include <iostream>
#include <cstdio>
#include <cmath>
#include <string>
#include <vector>

#include "methods.h"

using namespace std;

double origF(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}

double F(double x) {
    return pow(exp(1.0), 2 * pow(x, 2) - pow(x, 4) - 1);
}

int main(int argc, char *argv[]) {
    int n;
    double x;
    double eps;
    double delta = pow(10, -4);

    FILE *out_file = fopen(argv[1], "w");

    double x0 = 1.05;

    for (int i=1; i<=12; i++) {
        eps = pow(10, -i);
        x = ITER(x0, eps, n);
        fprintf(out_file, "%d %d\n", i, n);
    }
    fclose(out_file);

    return 0;
}
```

Файл `lab.cpp`:

```
#include <iostream>
#include <cstdio>
#include <cmath>
#include <string>
#include <vector>

#include "methods.h"

using namespace std;
```

```
double origF(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}

double F(double x) {
    return pow(exp(1.0), 2 * pow(x, 2) - pow(x, 4) - 1);
}

int main() {
    double x;
    double eps;
    double delta;

    int it;
    cout << "Enter epsilon presicion (decimal points): ";
    cin >> it; if (it <= 0) it = 1;
    eps = pow(10, -it);

    cout << "Enter delta presicion (decimal points): ";
    cin >> it; if (it <= 0) it = 1;
    delta = pow(10, -it);

    double x0 = 0.1;
    int n = 0;
    x = ITER(x0, eps, n);

     printf("\tFor epsilon = %lf and x0 = %lf: x = %lf with delta =
%lf, iter = %d\n", eps, x0, Round(x, delta), delta, n);

    return 0;
}
```

Файл `input_errors.cpp`:

```
#include <iostream>
#include <cstdio>
#include <cmath>
#include <string>
#include <vector>

#include "methods.h"

using namespace std;

double origF(double x) {
    return 2 * pow(x, 2) - pow(x, 4) - 1 - log(x);
}

double F(double x) {
    return pow(exp(1.0), 2 * pow(x, 2) - pow(x, 4) - 1);
}

int main(int argc, char *argv[]) {
    int n;
    double x;
    double eps = pow(4, -10);
```

```
    FILE *out_file = fopen(argv[1], "w");

    for (double x0=0; x0 < 2; x0 += 0.001) {
        x = ITER(x0, eps, n);
        fprintf(out_file, "%lf %d\n", x0, n);
    }

    return 0;
}
```

Файл `graph.py`:

```
import matplotlib
matplotlib.use("TkAgg")

import os
import sys
from matplotlib import pyplot as plt


if __name__ == "__main__":
    data_x = []
    data_y = []


    if not os.path.exists(sys.argv[1]):
        raise FileNotFoundError(f"No {sys.argv[1]} file!")
    file_nm = sys.argv[1]
    file = open(file_nm, "r")

    if sys.argv[2] == "precision":
        for line in file.readlines():
            if line:
                x, y = line.split()
                data_x.append(int(x))
                data_y.append(int(y))

        plt.plot(data_x, data_y, "bo", label="Result")
        plt.plot(data_x, data_y, "k--")

              plt.plot(data_x, list(map(lambda x: 2 * x**(1/2),
data_x)), "r--", label="Abstract f(x) = sqrt(x)")
        plt.title("Iterations ~ Precision")
        plt.xlabel("Precision, number of decimal places")
        plt.ylabel("Number of iterations")

    else:
        for line in file.readlines():
            if line:
                x, y = line.split()
                data_x.append(float(x))
                data_y.append(int(y))

            # plt.axvline(x=0.70711, color='red', linestyle='--',
label='f\'(x) = 0 \nx = max(f\'(x))')
        plt.plot(data_x, data_y, "bo", label="Result")
```

```
            # plt.plot(data_x, data_y, "k--")

            plt.title("Iterations ~ first approximation")
            plt.xlabel("Value of x0 -- first approximation")
            plt.ylabel("Number of iterations")

        plt.legend()
        plt.show()
```

Файл `Makefile`:

```
SRC_DIR = ${CURDIR}/src
WORK_DIR = ${SRC_DIR}/workdir
MAT_DIR = ${SRC_DIR}/Materials

lab: ${WORK_DIR}/lab
${WORK_DIR}/lab

precision: ${WORK_DIR}/precision.txt ${WORK_DIR}/graph.py
python3 ${WORK_DIR}/graph.py ${WORK_DIR}/precision.txt precision

input_errors: ${WORK_DIR}/input_errors.txt ${WORK_DIR}/graph.py
python3 ${WORK_DIR}/graph.py ${WORK_DIR}/input_errors.txt
input_errors

${WORK_DIR}/precision.txt: ${WORK_DIR}/precision.o
${WORK_DIR}/precision.o ${WORK_DIR}/precision.txt

${WORK_DIR}/input_errors.txt: ${WORK_DIR}/input_errors.o
${WORK_DIR}/input_errors.o ${WORK_DIR}/input_errors.txt

${WORK_DIR}/lab: ${WORK_DIR}/lab.cpp ${MAT_DIR}
g++ -g -I${MAT_DIR} ${WORK_DIR}/lab.cpp -o ${WORK_DIR}/lab

${WORK_DIR}/precision.o: ${WORK_DIR}/precision.cpp ${MAT_DIR}
g++ -g -I${MAT_DIR} ${WORK_DIR}/precision.cpp -o
${WORK_DIR}/precision.o

${WORK_DIR}/input_errors.o: ${WORK_DIR}/input_errors.cpp
${MAT_DIR}
g++ -g -I${MAT_DIR} ${WORK_DIR}/input_errors.cpp -o
${WORK_DIR}/input_errors.o

help:
@echo "Available options:"
@echo "   make [lab] -- calculate f(x) = 0 and show results"
@echo " make precision -- calculate f(x) for different epsilon for
given delta"
@echo "   make input_errors -- show input data sensitivity"
```